SIR ISSAC NEWTON
COLLEGE OF ENGINEERING AND TECHNOLOGY
(Approved by AICTE, New Delhi & Permenantly Affiliated to Anna University, Chennai)
An ISO 9001:2015 and 2(f) UGC Certified Institution, Accredited By NAAC With 'B ++ ' Grade
Velankanni Road, Pappakovil, Nagapattinam - 611 102 | Email : principal@sincet.ac.in | Web : www.sincet.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RECOMMENDER SYSTEMS

UNIT 1

INTRODUCTION TO RECOMMENDER SYSTEMS

UNIT-1: INTRODUCTION

UNIT I INTRODUCTION 6

Introduction and basic taxonomy of recommender systems - Traditional and non-personalized Recommender Systems - Overview of data mining methods for recommender systems- similarity measures- Dimensionality reduction – Singular Value Decomposition (SVD)

Suggested Activities:

- Practical learning – Implement Data similarity measures.
- External Learning – Singular Value Decomposition (SVD) applications

Suggested Evaluation Methods:

- Quiz on Recommender systems.
- Quiz of python tools available for implementing Recommender systems

INTRODUCTION:

Recommender systems, also known as recommendation systems or engines, are a type of software application designed to provide personalized suggestions or recommendations to users. These systems are widely used in various online platforms and services to help users discover items or content of interest. Recommender systems leverage data about users' preferences, behaviors, and interactions to generate accurate and relevant recommendations.

WHAT ARE RECOMMENDER SYSTEMS?

Recommender systems are sophisticated algorithms designed to provide product-relevant suggestions to users. Recommender systems play a paramount role in enhancing user experiences on various online platforms, including e-commerce websites, streaming services, and social media.

Essentially, recommender systems aim to analyze user data and behavior to make tailored recommendations.

- **Data collection:** Recommender systems start by gathering data on user interactions, preferences, and behaviors. This data can include past purchases, browsing history, ratings, and social connections**.**
- **Data processing:** Once collected, they process the data to extract meaningful patterns and insights. This involves techniques like data cleaning, transformation, and feature engineering**.**
- **Algorithm selection:** Depending on the specific platform and its data, a specific recommender algorithm is applied to generate recommendations. Common types include collaborative filtering, content-based filtering, and hybrid methods**.**
- **User profiling:** Using historical data, recommender systems create user profiles. These represent their preferences, interests, and behavior, allowing the system to understand individual tastes**.**
- **Item profiling:** Similarly, items or content available on the platform are also profiled based on their characteristics. Think of attributes like genres, keywords, or product features**.**
- **Recommendation generation:** The next step involves algorithms matching user profiles with item profiles. For example, collaborative filtering identifies users with similar preferences and recommends items liked by others with similar profiles. Content-based filtering recommends items based on the attributes of items users have previously interacted with.

- **Ranking and presentation:** Finally, the recommended items are ranked based on their relevance to the user. The top-ranked items are then presented to the user through interfaces like recommendation lists, personalized emails, or pop-up suggestions.

what is the basic principle that underlies the working of recommendation algorithms?

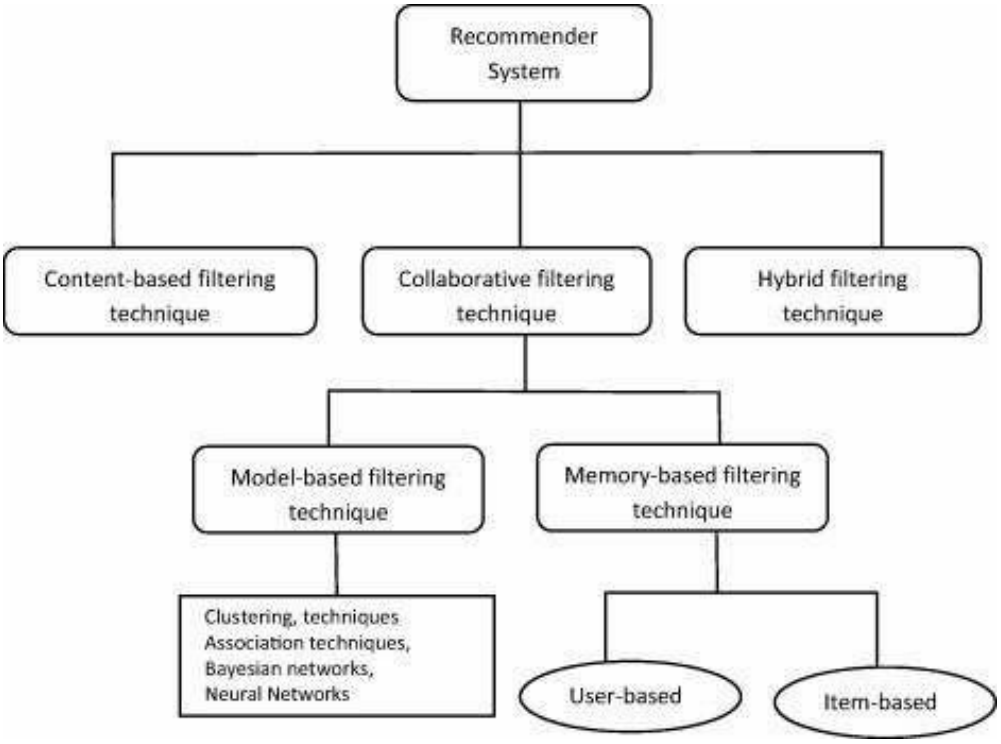
The basic principle of recommendations is that significant dependencies exist between user and item-centric activity. For example, a user who is interested in a historical documentary is more likely to be interested in another historical documentary or an educational program, rather than in an action movie. In many cases, various categories of items may show significant correlations, which can be leveraged to make more accurate recommendations.

Alternatively, the dependencies may be present at the finer granularity of individual items rather than categories. These dependencies can be learned in a data-driven manner from the ratings matrix, and the resulting model is used to make predictions for target users.

The larger the number of rated items that are available for a user, the easier it is to make robust predictions about the future behavior of the user. Many different learning models can be used to accomplish this task. For example, the collective buying or rating behavior of various users can be leveraged to create partners of similar users that are interested in similar products. The interests and actions of these cohorts can be leveraged to make recommendations to individual members of these cohorts.

The above-mentioned description is based on a very simple family of recommendation algorithms, referred to as neighborhood models. This family belongs to a broader class of models, referred to as collaborative filtering. The term "collaborative filtering" refers to the use of ratings from multiple users in a collaborative way to predict missing ratings. In practice, recommender systems can be more complex and data-rich, with a wide variety of auxiliary data types. For example, in content-based recommender systems, the content plays a primary role in the recommendation process, in which the ratings of users and the attribute descriptions of items are leveraged in order to make predictions. The basic idea is that user interests can be modeled on the basis of properties (or attributes) of the items they have rated or accessed in the past. A different framework is that of knowledge-based systems, in which users interactively specify their interests, and the user specification is combined with domain knowledge to provide recommendations. In advanced models, contextual data, such as temporal information, external knowledge, location information, social information, or network information, may be used.

There are several types of recommender systems, each with its own approach to generating recommendations. The basic taxonomy of recommender systems includes:



a. Content-Based Recommender Systems:

- **Overview:** Content-based systems recommend items based on the features of the items themselves and the preferences expressed by the user.
- **Key Components:** The system analyzes the content of items and creates user profiles based on the features of items the user has liked or interacted with in the past.

b. Collaborative Filtering Recommender Systems:

- **Overview:** Collaborative filtering relies on user-item interactions and recommendations from other users with similar preferences to make predictions for a target user.
- **Types:**
  - **User-Based Collaborative Filtering: Recommends** items based on the preferences of users who are similar to the target user.
  - **Item-Based Collaborative Filtering: Recommends** items that are similar to those the user has liked or interacted with in the past.

c. Hybrid Recommender Systems:

- **Overview:** Hybrid systems combine multiple recommendation techniques to overcome the limitations of individual methods, providing more accurate and diverse recommendations.
- **Types:**

o **Weighted Hybrid:** Assigns different weights to recommendations from different methods and combines them.

o **Switching Hybrid:** Switches between different recommendation methods based on certain conditions or user interactions.

There are some more types of recommender systems. They are:

i. **Matrix Factorization Recommender Systems:**

- **Overview:** Matrix factorization models decompose the user-item interaction matrix into latent factors, allowing the system to make predictions based on these factors.

- **Example:** Singular Value Decomposition (SVD) and Alternating Least Squares (ALS) are common matrix factorization techniques used in recommender systems.

ii. **Context-Aware Recommender Systems:**

- **Overview:** Context-aware systems take into account additional contextual information, such as time, location, or user activity, to enhance the relevance of recommendations.

- **Example:** Recommending different movies based on the time of day or suggesting nearby restaurants based on a user's location.

iii. **Knowledge-Based Recommender Systems:**

- **Overview:** Knowledge-based systems recommend items by taking into account explicit knowledge about user preferences, requirements, and item characteristics.

- **Example:** Recommending educational courses based on a user's career goals and academic background.

iv. **Deep Learning-Based Recommender Systems:**

- **Overview:** Deep learning techniques, such as neural networks, are employed to model complex patterns and dependencies in user-item interactions for more accurate recommendations.

- **Example:** Neural collaborative filtering models that use embeddings to represent users and items.

GOALS OF RECOMMENDER SYSTEMS

The two primary models are as follows:

1. **Prediction version of problem:** The first approach is to predict the rating value for a user-item combination. It is assumed that training data is available, indicating user preferences for items. For m users and n items, this corresponds to an incomplete $m \times n$ matrix, where the specified (or observed) values are used for training. The missing (or unobserved) values are predicted using this training model. This problem is also referred to as the matrix completion problem because we have an incompletely specified matrix of values, and the remaining values are predicted by the learning algorithm.

2. **Ranking version of problem:** In practice, it is not necessary to predict the ratings of users for specific items in order to make recommendations to users. Rather, a merchant may wish to recommend the top-k items for a particular user, or determine the top-k users to target for a particular item. The determination of the top-k items is more common than the determination of top-k users, although the methods in the two cases are exactly analogous.

Increasing product sales is the primary goal of a recommender system. Recommender systems are, after all, utilized by merchants to increase their profit. By recommending carefully selected items to users, recommender systems bring relevant items to the attention of users. This increases the sales volume and profits for the merchant.

In order to achieve the broader business-centric goal of increasing revenue, the **common operational and technical goals of recommender systems are as follows:**

- **Relevance:** The most obvious operational goal of a recommender system is to recommend items that are relevant to the user at hand. Users are more likely to consume items they find interesting. Although relevance is the primary operational goal of a recommender system, it is not sufficient in isolation. Therefore, we discuss several secondary goals below, which are not quite as important as relevance but are nevertheless important enough to have a significant impact
- **Novelty:** Recommender systems are truly helpful when the recommended item is something that the user has not seen in the past. For example, popular movies of a preferred genre would rarely be novel to the user. Repeated recommendation of popular items can also lead to reduction in sales diversity.
- **Serendipity:** A related notion is that of fate, wherein the items recommended are somewhat unexpected, and therefore there is a modest element of lucky discovery, as opposed to obvious recommendations. Serendipity is different from novelty in that the recommendations are truly surprising to the user, rather than simply something they did not know about before. It may often be the case that a particular user may only be consuming items of a specific type, although a latent interest in items of other types may exist which the user might themselves find surprising. Unlike novelty, serendipitous methods focus on discovering such recommendations.
- **Increasing recommendation diversity:** Recommender systems typically suggest a list of top-k items. When all these recommended items are very similar, it increases the risk that the user might not like any of these items. On the other hand, when the recommended list contains items of different types, there is a greater chance that the user might like at least one of these items. Diversity has the benefit of ensuring that the user does not get bored by repeated recommendation of similar items.

Examples of Recommender Systems

In today's digital age, we are bombarded with vast information and choices. **From online shopping to streaming services**, it can be overwhelming to navigate through the plethora of options available. This is where recommender systems come in – they help us make sense of the endless choices by suggesting relevant options based on our interests and preferences.

1. **Netflix**

   Netflix's recommendation engine is perhaps the most well-known and widely used **recommender system**. It uses an algorithm to analyze a user's viewing history, rating, and search behavior to suggest movies and TV shows that the user is likely to enjoy. The algorithm takes into account the genre, the actors, the director, and other factors to make personalized recommendations for each user.

2. **Amazon**

   Amazon's recommendation engine suggests products based on a user's purchase history, search history, and browsing behavior. It makes personalized recommendations based on the user's prior purchases, products viewed, and items added to their shopping cart.

3. **Spotify**

   Spotify's music recommendation system suggests songs, playlists, and albums depending on a user's listening history, liked songs, and search history. It tailors recommendations based on the user's listening habits, favorite genres, and favorite artists.

4. **YouTube**

   YouTube's recommendation engine suggests videos based on a user's viewing history, liked videos, and search history. The algorithm considers factors such as the user's favourite channels, the length of time spent watching a video, and other viewing habits to make personalized recommendations.

5. **LinkedIn**

   LinkedIn's recommendation engine suggests jobs, connections, and content based on a user's profile, skills, and career history. To make personalized recommendations, the algorithm takes the user's job title, industry, and location.

6. **Zillow**

   Zillow's recommendation system suggests real estate properties depend on a user's search history and preferences. Users can receive personalized recommendations based on their budget, location, and desired features.

7. **Airbnb**

   Airbnb's recommendation system suggests accommodations based on a user's search history, preferences, and reviews. Personal recommendations are made based on factors such as the user's travel history, location, and desired amenities.

8. **Uber**

   Uber's recommendation system suggests ride options created on a user's previous rides and preferred options. When recommending rides, the algorithm considers factors such as the user's preferred vehicle type, location, and other preferences.

9. **Google Maps**

   Google Maps' recommendation system suggests places to visit, eat, and shop based on a user's search history and location. Personalized recommendations are generated based on factors such as the user's location, time of day, and preferences.

10. **Goodreads**

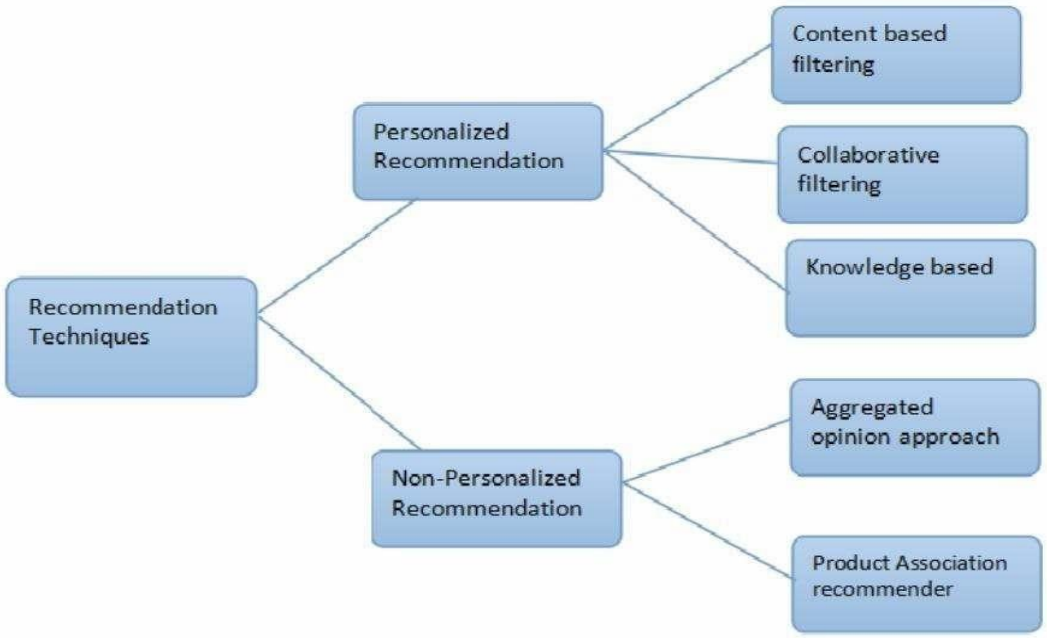
UNIT-1: INTRODUCTION

Goodreads' recommendation engine suggests books centred on a user's reading history, ratings, and reviews. To provide personalized recommendations, the algorithm considers factors such as the user's reading habits, genres, and favorite authors.

From online shopping to entertainment and travel. These systems have significantly improved the user experience by suggesting relevant options based on our interests and preferences. The success of these real-world examples showcases the power and effectiveness of **recommender systems** in various industries. With advancements in artificial intelligence, recommender systems are expected to become even more accurate and personalized in the future.

PERSONALIZED AND NON-PERSONALIZED RECOMMENDER SYSTEMS:

personalized and non-personalized recommender systems are two broad categories of recommendation engines that differ in their approach to generating recommendations. While personalized systems tailor recommendations to individual users based on their preferences and behavior, non-personalized systems provide the same recommendations to all users, regardless of their individual characteristics. Let's delve into each category:



A. Traditional Recommender Systems:

- **Overview:** Traditional recommender systems typically use explicit input features or rules to generate recommendations. These systems often rely on general attributes of items or users, and their recommendations are not personalized to the specific preferences or behavior of individual users.
- **Example Features:**
  - Genre of a movie
  - Author of a book
  - Popularity or overall ratings of items
- **Methodology:**

- Recommendations are made based on fixed criteria or predetermined rules.
- Users receive the same recommendations regardless of their unique preferences.
-

- **Advantages:**
  - Simplicity and ease of implementation.
  - Less reliance on individual user data.
  - Suitable for scenarios where personalization is not a critical factor.

B. Non-personalized Recommender Systems:

- **Overview:** Non-personalized recommender systems provide the same set of recommendations to all users, without considering individual user preferences or behaviors. These systems often focus on providing popular or trending items that are likely to appeal to a broad audience.
- **Examples:**
  - "Top 10" lists or rankings
  - Bestsellers
  - Most viewed items
- **Methodology:**
  - Recommendations are based on aggregate data, such as overall popularity or global trends.
  - All users receive identical recommendations.
- **Advantages:**
  - Easy to implement and computationally efficient.
  - Applicable in scenarios where personalization is not feasible or necessary.
  - Can be effective for new users or when limited user data is available.

- While personalized and non-personalized recommender systems have their advantages, they may lack the ability to provide highly relevant and tailored recommendations that reflect individual user preferences. As a result, these systems are often contrasted with personalized recommender systems, which leverage user-specific data to deliver more accurate and targeted suggestions.

PERSONALIZED RECOMMENDER SYSTEMS

Personalized recommendation systems are designed to provide tailored recommendations to individual users based on their past behavior, preferences, and demographic information.

Based on the user's data such as purchases or ratings, personalized recommenders try to understand and predict what items or content a specific user is likely to be interested in. In that way, every user will get customized recommendations.

what makes a good recommendation?

- Is personalized (relevant to that user),
- Is diverse (includes different user interests),
- Doesn't recommend the same items to users for the second time, and
- Recommends available products at the right time.

There are a few types of personalized recommendation systems, including content-based filtering, collaborative filtering, and hybrid recommenders.

TYPES OF PERSONALIZED RECOMMENDER SYSTEMS

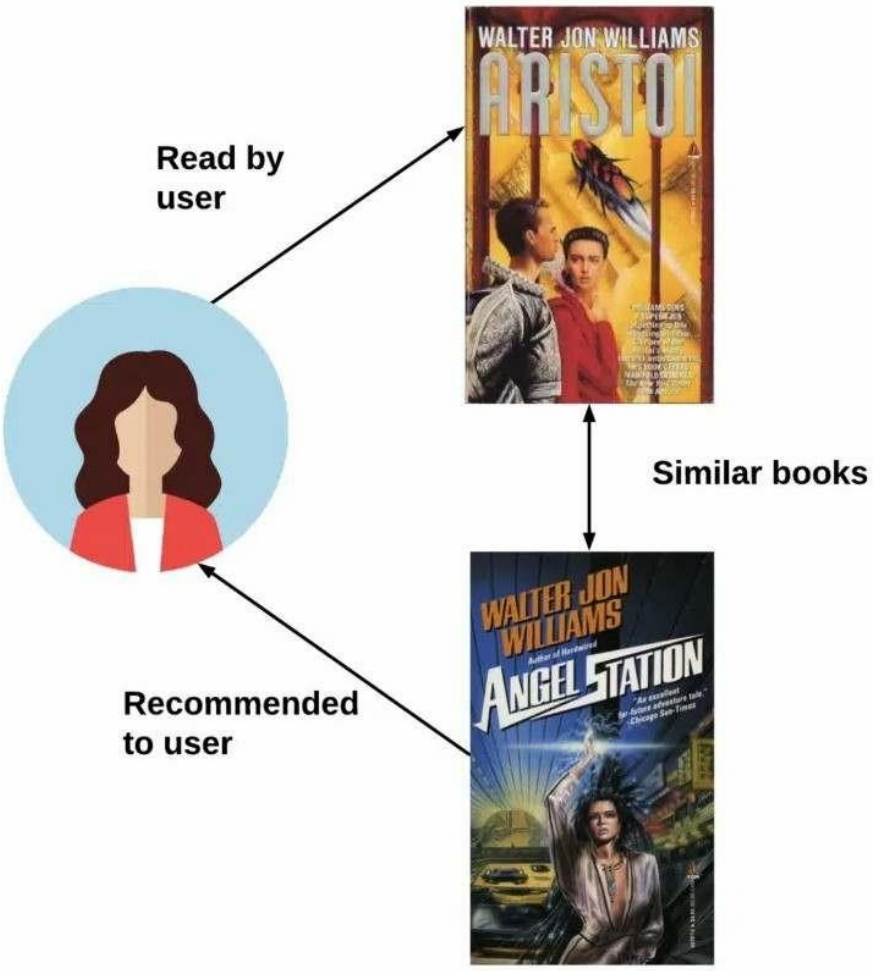
Personalized recommender systems can be categorized into several types, each with its own methods and techniques for providing tailored recommendations.

These include:

- Content-based filtering,
- Collaborative filtering, and
- Hybrid recommenders.

CONTENT-BASED FILTERING

- Content-based recommender systems use items or user metadata to create specific recommendations. To do this, we look at the user's purchase history.
- For example, if a user has already read a book from one author or a product from a certain brand, you assume that they have a preference for that author or that brand. Also, there is a probability that they will buy a similar product in the future.



Let's assume that Jenny loves sci-fi books and her favorite writer is Walter Jon Williams. If she reads the Aristoi book, then her recommended book will be Angel Station, also a sci-fi book written by Walter Jon Williams.

Pros of the content-based approach

The content-based approach is one of the common techniques used in personalized recommendation systems. It has its advantages and disadvantages, which are important to consider when deciding to implement this approach.

Advantages

- Less cold-start problem: Content-based recommendations can effectively address the "cold-start" problem, allowing new users or items with limited interaction history to still receive relevant recommendations.
- Transparency: Content-based filtering allows users to understand why a recommendation is made because it's based on the content and attributes of items they've previously interacted with.
- Diversity: Considering various attributes, content-based systems can provide diverse recommendations. For example, in a movie recommendation system, recommendations can be based on genre, director, and actors.

- Reduced data privacy concerns: Since content-based systems primarily use item attributes, they may not require as much user data, which can mitigate privacy concerns associated with collecting and storing user data.

Disadvantages of the content-based approach

- **The "Filter bubble**": Content filtering can recommend only content similar to the user's past preferences. If a user reads a book about a political ideology and books related to that ideology are recommended to them, they will be in the "bubble of their previous interests".
- **Limited serendipity**: Content-based systems may have limited capability to recommend items that are outside a user's known preferences.
- In the first case scenario, 20% of items attract the attention of 70-80% of users and 70-80% of items attract the attention of 20% of users. The recommender's goal is to introduce other products that are not available to users at first glance.
- In the second case scenario, content-based filtering recommends products that are fitting content-wise, yet very unpopular (i.e. people don't buy those products for some reason, for example, the book is bad even though it fits thematically).
- **Over-specialization**: If the content-based system relies too heavily on a user's past interactions, it can recommend items that are too similar to what the user has already seen or interacted with, potentially missing opportunities for diversification.

COLLABORATIVE FILTERING

- Collaborative filtering is a popular technique used to provide personalized recommendations to users based on the behavior and preferences of similar users.
- The fundamental idea behind collaborative filtering is that users who have interacted with items in similar ways or have had similar preferences in the past are likely to have similar preferences in the future, too.
- Collaborative filtering relies on the collective wisdom of the user community to generate recommendations.

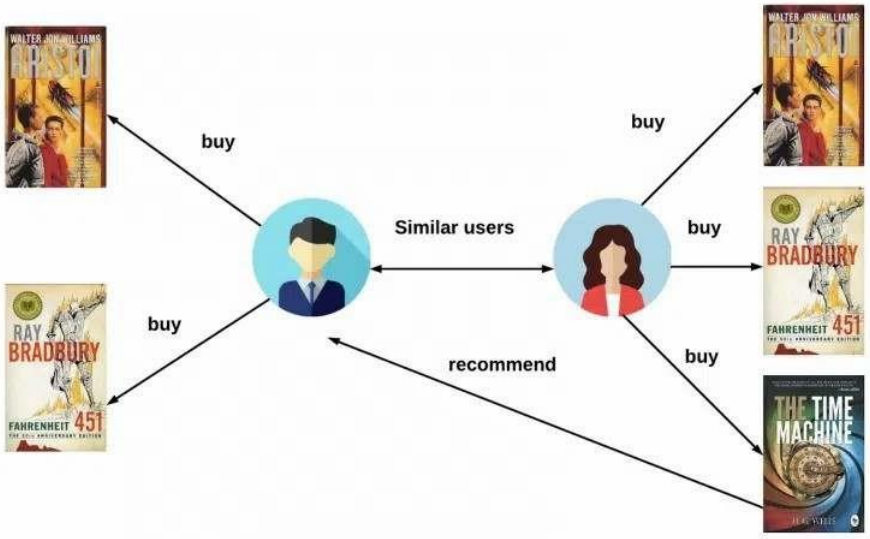
There are two main types of collaborative filtering: memory-based and model-based.

Memory-based recommenders

- Memory-based recommenders rely on the direct similarity between users or items to make recommendations.
- Usually, these systems use raw, historical user interaction data, such as user-item ratings or purchase histories, to identify similarities between users or items and generate personalized recommendations.
- The biggest disadvantage of memory-based recommenders is that they require a lot of data to be stored and comparing every item/user with every item/user is extremely computationally demanding.

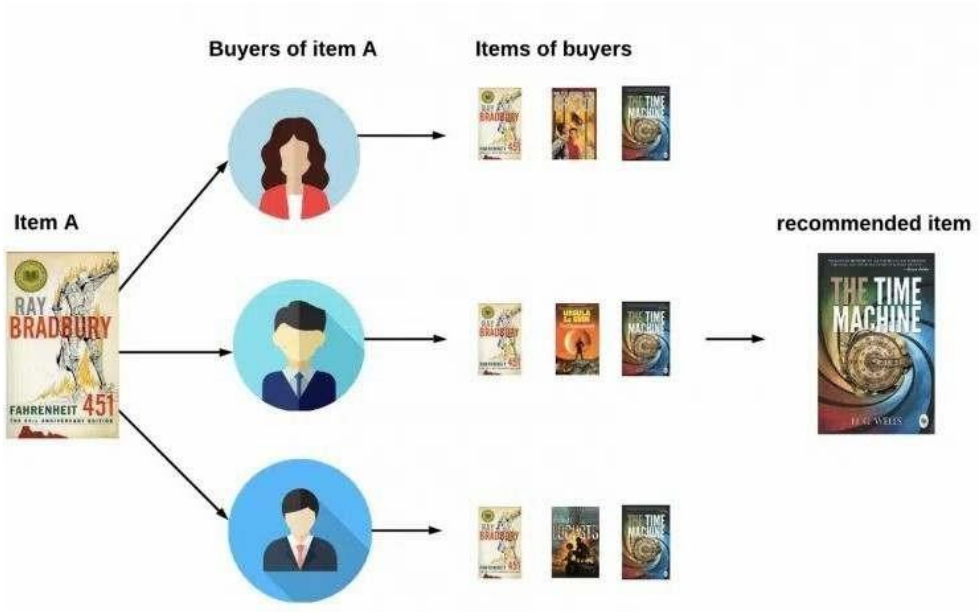
- Memory-based recommenders can be categorized into two main types user-based and item-based collaborative filtering.

A user-based collaborative filtering recommender system



- With the used-based approach, recommendations to the target user are made by identifying other users who have shown similar behavior or preferences. This translates to finding users who are most similar to the target user based on their historical interactions with items. This could be "users who are similar to you also liked…" type of recommendations.
- But if we say that users are similar, what does that mean?
- Let's say that Jenny and Tom both love sci-fi books. This means that, when a new sci-fi book appears and Jenny buys that book, that same book will be recommended to Tom, since he also likes sci-fi books.

An item-based collaborative filtering recommender system



- In item-based collaborative filtering, recommendations are made by identifying items that are similar to the ones the target user has already interacted with.

- The idea is to find items that share similar user interactions and recommend those items to the target user. This can include "users who liked this item also liked…" type of recommendations.
- To illustrate with an example, let's assume that John, Robert, and Jenny highly rated sci-fi books Fahrenheit 451 and The Time Machine, giving them 5 stars. So, when Tom buys Fahrenheit 451, the system automatically recommends The Time Machine to him because it has identified it as similar based on other users' ratings.

How to calculate user-user and item-item similarities?

- Unlike the content-based approach where metadata about users or items is used, in the collaborative filtering memory-based approach we are looking at the user's behavior e.g. whether the user liked or rated an item or whether the item was liked or rated by a certain user.
- For example, the idea is to recommend Robert the new sci-fi book. Let's look at the steps in this process:
- **Create a user-item-rating matrix.**
- **Create a user-user similarity matrix:** Cosine similarity is calculated (alternatives: adjusted cosine similarity, Pearson similarity, Spearman rank correlation) between every two users. This is how we get a user-user matrix. This matrix is smaller than the initial user-item-rating matrix.

$$similarity(A,B) = \frac{A \cdot B}{\|A\| \times \|B\|} = \frac{\sum_{i=1}^{n} A_i \times B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \times \sqrt{\sum_{i=1}^{n} B_i^2}}$$

- **Look up similar users**: In the user-user matrix, we observe users that are most similar to Robert.
- **Candidate generation**: When we find Robert's most similar users, we look at all the books these users read and the ratings they gave them.
- **Candidate scoring:** Depending on the other users' ratings, books are ranked from the ones they liked the most, to the ones they liked the least. The results are normalized on a scale from 0 to 1.
- **Candidate filtering: We** check if Robert has already bought any of these books and eliminate those he already read.
- The item-item similarity calculation is done in an identical way and has all the same steps as user-user similarity.

Model-based recommenders

- Model-based recommenders make use of machine learning models to generate recommendations.

- These systems learn patterns, correlations, and relationships from historical user-item interaction data to make predictions about a user's preferences for items they haven't interacted with yet.
- There are different types of model-based recommenders, such as matrix factorization, Singular Value Decomposition (SVD), or neural networks.
- However, matrix factorization remains the most popular one, so let's explore it a bit further.

Matrix factorization

- Matrix factorization is a mathematical technique used to decompose a large matrix into the product of multiple smaller matrices.
- In the context of recommender systems, matrix factorization is commonly employed to uncover latent patterns or features in user-item interaction data, allowing for personalized recommendations. Latent information can be reported by analyzing user behavior.
- If there is feedback from the user, for example – they have watched a particular movie or read a particular book and have given a rating, that can be represented in the form of a matrix. In this case,
    - Rows represent users,
    - Columns represent items, and
    - The values in the matrix represent user-item interactions (e.g., ratings, purchase history, clicks, or binary preferences).

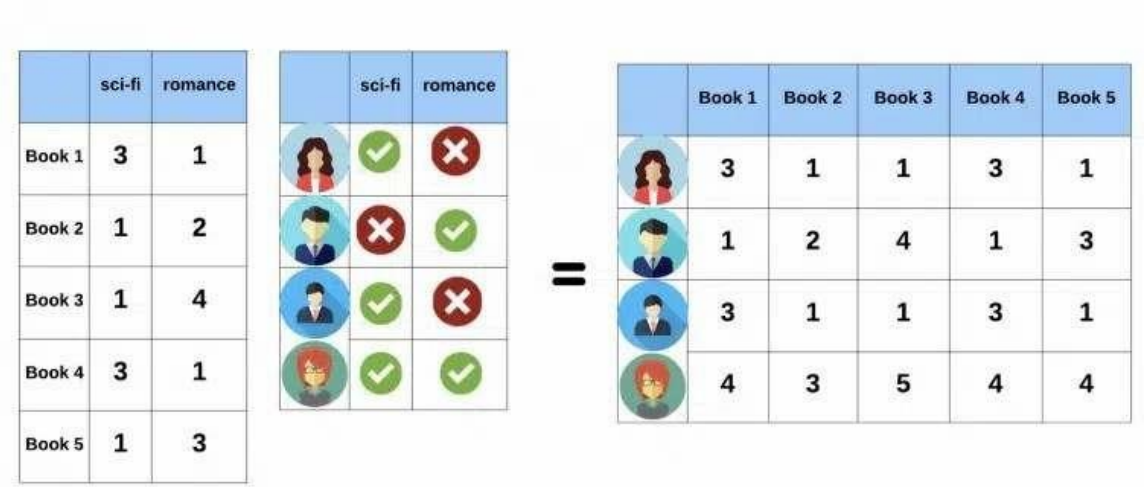
Since it's almost impossible for the user to rate every item, this matrix will have many unfilled values. This is called sparsity.

The matrix factorization process

Matrix factorization aims to approximate this interaction matrix by factorizing it into two or more lower-dimensional matrices:

- User latent factor matrix (U), which contains information about users and their relationships with latent factors.
- Item latent factor matrix (V), which contains information about items and their relationships with latent factors.

The rating matrix is a product of two smaller matrices – the item-feature matrix and the user-feature matrix. The higher the score in the matrix, the better the match between the item and the user.



The matrix factorization process includes the following steps:

- Initialization of random user and item matrix,
- The ratings matrix is obtained by multiplying the user and the transposed item matrix,
- The goal of matrix factorization is to minimize the loss function (the difference in the ratings of the predicted and actual matrices must be minimal). Each rating can be described as a dot product of a row in the user matrix and a column in the item matrix.

Minimization of loss function

$$\min_{Q*,P*} \sum_{(u,i)\in K} (r_{ui} - P_u^T Q_i)^2 + \lambda(||Q_i||^2 + ||P_u||^2)$$

- Where K is a set of (u, i) pairs, r(u, i) is the rating for item i by user u and λ is a regularization term (used to avoid overfitting).
- In order to minimize loss function we can apply Stochastic Gradient Descent (SGD) or Alternating Least Squares (ALS). Both methods can be used to incrementally update the model as new rating comes in. SGD is faster and more accurate than ALS.

Advantages of collaborative filtering

- **Effective personalization**: Collaborative filtering is highly effective in providing personalized recommendations to users. It takes into account the behavior and preferences of similar users to suggest items that a particular user is likely to enjoy.
- **No need for item attributes**: Collaborative filtering works solely based on user-item interactions, making it applicable to a wide range of recommendation scenarios where item features may be sparse or unavailable. This is especially useful in content-rich platforms.

- **Serendipitous (unanticipated) discoveries**: Collaborative filtering can introduce users to items they might not have discovered otherwise. By analyzing user behaviors and identifying patterns across the user community, collaborative filtering can recommend items that align with a user's tastes but may not be immediately obvious to them.

Disadvantages of collaborative filtering

It's important to note that while collaborative filtering offers these and other advantages, it also has its limitations, including:

The "cold-start" problem:

- User cold start occurs when a new user joins the system without any prior interaction history. Collaborative filtering relies on historical interactions to make recommendations, so it can't provide personalized suggestions to new users who start with no data.
- Item cold start happens when a new item is added, and there's no user interaction data for it. Collaborative filtering has difficulty recommending new items since it lacks information about how users have engaged with these items in the past.
- **Sensitivity to sparse data**: Collaborative filtering depends on having enough user-item interaction data to provide meaningful recommendations. In situations where data is sparse and users interact with only a small number of items, collaborative filtering may struggle to find useful patterns or similarities between users and items.
- **Potential for popularity bias:** Collaborative filtering tends to recommend popular items more frequently. This can lead to a "rich get richer" phenomenon, where already popular items receive even more attention, while niche or less-known items are overlooked.
- **To address these and other limitations**, recommendation systems often use hybrid approaches that combine collaborative filtering with content-based methods or other techniques to improve recommendation quality in the long run.

HYBRID RECOMMENDERS

- Hybrid recommendation systems combine multiple recommendation techniques or approaches to provide more accurate, diverse, and effective personalized recommendations.
- They are particularly valuable in real-world recommendation scenarios because they can provide more robust, accurate, and adaptable recommendations.
- The choice of which hybrid approach to use depends on the specific requirements and constraints of the recommendation system and the nature of the available data.

Pros of hybrid recommenders

Some of the most common advantages of hybrid recommenders include:

- **Improved recommendation quality**: Hybrid recommenders leverage multiple recommendation techniques, combining their strengths to provide more accurate and diverse recommendations. This often results in higher recommendation quality

compared to individual methods, benefiting users by offering more relevant suggestions.

- **Enhanced robustness and flexibility**: Hybrid models are often more robust in handling various recommendation scenarios. They can adapt to different data characteristics, user behaviors, and recommendation challenges. This flexibility is valuable in real-world recommendation systems.
- **Addressing common recommendation limitations**: Hybrid recommenders can mitigate the limitations of individual recommendation techniques. For example, they can overcome the "cold-start" problem for new users and items by incorporating content-based recommendations, providing serendipitous suggestions, and reducing popularity bias.

Cons of hybrid recommenders

- Just like all other recommenders systems, hybrid recommenders have their downsides, too. Some include:

- **Increased complexity and development effort**: Implementing and maintaining hybrid recommendation systems can be more complex and resource-intensive. It requires expertise in multiple recommendation techniques and careful integration of these methods.
- **Data and computational demands**: Hybrid models often require more data and computational resources because they use multiple recommendation algorithms. This can be challenging, especially in large-scale systems with vast user-item interactions and a diverse catalog of items.
- **Tuning and parameter sensitivity**: Hybrid recommenders may involve a greater number of parameters and hyperparameters that need to be fine-tuned. Yet, ensuring optimal parameter settings for each recommendation component can be challenging and time-consuming.
- **While hybrid recommenders** offer significant advantages in terms of recommendation quality and versatility, you should carefully consider the trade-offs and resource requirements when deciding which system to implement.
- This is the best way to ensure that the benefits of hybridization outweigh the added complexity and costs.

EVALUATION METRICS FOR RECOMMENDER SYSTEMS

- To assess the performance and effectiveness of recommender systems, you have to take into consideration certain evaluation metrics.
- They can help you measure how well a recommendation algorithm or model is performing and provide insights into its strengths and weaknesses.
- There are several categories of evaluation metrics, depending on the specific aspect of recommendations being assessed.

Some common evaluation metrics include:

- **Accuracy metrics** assess the accuracy of the recommendations made by a system in terms of how well they match the user's actual preferences or behavior. Here we have

Mean Absolute Error (MAE), Root Mean Square Error (RMSE), or Mean Squared Logarithmic Error (MSLE).

- **Ranking metrics** evaluate how well a recommender system ranks items for a user, especially in top-N recommendation scenarios. Think of hit rate, average reciprocal hit rate (ARHR), cumulative hit rate, or rating hit rate.
- **Diversity metrics** assess the diversity of recommended items to ensure that recommendations are not overly focused on a narrow set of items. These include Intra-List Diversity or Inter-List Diversity.
- **Novelty metrics** evaluate how well a recommender system introduces users to new or unfamiliar items. Catalog coverage and item popularity belong to this category.
- **Serendipity metrics** assess the system's ability to recommend unexpected but interesting items to users – surprise or diversity are looked at in this case.

You can also choose to look at some **business metrics such as conversion rate, click-through rate (CTR), or revenue impact. But, ultimately, the best way to do an online evaluation of your recommender system is through A/B testing**.

OVERVIEW OF DATA MINING METHODS:

Recommender Systems (RS) typically apply techniques and methodologies from other neighboring areas– such as Human Computer Interaction (HCI) or Information Retrieval (IR). However, most of these systems bear in their core an algorithm that can be understood as a particular instance of a Data Mining (DM) technique

Data mining methods play a crucial role in building effective recommender systems by extracting patterns and insights from large datasets. One key aspect of recommender systems involves measuring similarity between users, items, or both. Let's explore an overview of data mining methods for recommender systems and common similarity measures:

Data Mining Methods for Recommender Systems:

1. **Association Rule Mining:**

- **Overview:** Association rule mining identifies relationships or patterns in user-item interactions. It helps discover associations between items that are frequently co-purchased or co-viewed.
- **Application:** Generating recommendations based on association rules, e.g., "Users who bought X also bought Y."

2. **Clustering Algorithms:**

- **Overview:** Clustering methods group users or items with similar characteristics. Users or items within the same cluster are likely to share common preferences.
- **Application:** Recommending items popular within a user's cluster, assuming similar preferences within the group.

3. **Classification Algorithms:**

- **Overview:** Classification models predict user preferences for items based on historical interactions. These models can be trained to classify items as relevant or irrelevant to a user.
- **Application:** Providing recommendations by predicting user preferences for items not yet interacted with.

4. **Matrix Factorization:**

- **Overview:** Matrix factorization techniques decompose the user-item interaction matrix into latent factors, capturing hidden patterns and relationships. Singular Value Decomposition (SVD) and Alternating Least Squares (ALS) are common matrix factorization methods.
- **Application:** Predicting missing values in the user-item matrix to recommend items a user might like.

5. **Deep Learning Models:**

- **Overview:** Deep learning models, such as neural networks, can capture complex patterns in user-item interactions. Neural collaborative filtering is an example where embeddings are used to represent users and items.
- **Application:** Learning intricate user-item relationships for more accurate and personalized recommendations.

Similarity Measures:

Different data types require different functions to measure the similarity of data points. Diffentiation between unary, binary and quantitative data helps with most problems. Unary data could be the number of likes for a blog post. Binary data could be likes **and** dislikes of a video and quantitative data could be rating provided like 4/10 stars or similar. The following table summarises which similarity functions are suitable for different data types.

Data type	Similarity function
Unary data	Jaccard Similarity
Binary data	Jaccard Similarity
Quantitative data	Pearson or cosine similarity

1. **Cosine Similarity:**

- **Definition:** Measures the cosine of the angle between two vectors, representing users or items, in a multidimensional space.
- Cosine similarity is a measure used to determine the similarity between two non-zero vectors in a vector space. It calculates the cosine of the angle between the vectors, representing their orientation and similarity.

$$CosineSimilarity(A, B) = \frac{A \cdot B}{\|A\| \, \|B\|}$$

- A · B denotes the dot product of vectors A and B, which is the sum of the element-wise multiplication of their corresponding components.
- ||A|| represents the Euclidean norm or magnitude of vector A, calculated as the square root of the sum of the squares of its components.
- ||B|| represents the Euclidean norm or magnitude of vector B.

The resulting value ranges from -1 to 1, where 1 indicates that the vectors are in the same direction (i.e., completely similar), -1 indicates they are in opposite directions (i.e., completely dissimilar), and 0 indicates they are orthogonal or independent (i.e., no

similarity). It is particularly useful in scenarios where the magnitude of the vectors is not significant, and the focus is on the direction or relative orientation of the vectors.

Dimensionality Independence: It is not affected by the magnitude or length of vectors. It solely focuses on the direction or orientation of the vectors. This property makes it valuable when dealing with high-dimensional data or sparse vectors, where the magnitude of the vectors may not be as informative as their relative angles or orientations.

Sparse Data: It is particularly effective when working with sparse data, where vectors have many zero or missing values. In such cases, the non-zero elements play a crucial role in capturing the meaningful information and similarity between vectors.

- **Application**: In **recommender systems**, cosine similarity can be used to measure the similarity between user preferences or item characteristics, aiding in generating personalised recommendations based on similar user preferences or item profiles.

2. **Pearson Correlation Coefficient:**
   - **Definition:** Measures linear correlation between two variables, providing a measure of the strength and direction of a linear relationship.
   - The Pearson correlation coefficient, also known as Pearson's correlation or simply correlation coefficient, is a statistical measure that quantifies the linear relationship between two variables. It measures how closely the data points of the variables align on a straight line, indicating the strength and direction of the relationship.

$$r(A,B) = \frac{\sum((A-\mu_A)*(B-\mu_B))}{\sqrt{\sum(A-\mu_A)^2}*\sqrt{\sum(B-\mu_B)^2}}$$

   The Pearson correlation coefficient is denoted by the symbol "r" and takes values between -1 and 1. The coefficient value indicates the following:
   - r = 1: Perfect positive correlation. The variables have a strong positive linear relationship, meaning that as one variable increases, the other variable also increases proportionally.
   - r = -1: Perfect negative correlation. The variables have a strong negative linear relationship, meaning that as one variable increases, the other variable decreases proportionally.
   - r = 0: No linear correlation. There is no linear relationship between the variables. They are independent of each other.
   - **Application:** Evaluating how well users' preferences align, especially in scenarios with numerical ratings.

3. **Jaccard Similarity:**
   - **Definition:** Measures the intersection over the union of sets, quantifying the similarity between two sets.
   - It calculates the size of the intersection of the sets divided by the size of their union. The resulting value ranges from 0 to 1, where 0 indicates no similarity and 1 indicates complete similarity.

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

- In other words, to calculate the Jaccard similarity, you need to determine the common elements between the sets of interest and divide it by the total number of distinct elements across both sets.
- In other words, to calculate the Jaccard similarity, you need to determine the common elements between the sets of interest and divide it by the total number of distinct elements across both sets.
- It is useful because it provides a straightforward and intuitive measure to quantify the similarity between sets. Its simplicity makes it applicable in various domains and scenarios.
- **Here are some key reasons for its usefulness:**
    - **Set Comparison:** It enables the comparison of sets without considering the specific elements or their ordering. It focuses on the presence or absence of elements, making it suitable for cases where the structure or attributes of the elements are not important or would need additional feature engineering, which would slow down the system.
    - **Scale-Invariant**: It remains unaffected by the size of the sets being compared. It solely relies on the intersection and union of sets, making it a robust measure even when dealing with sets of different sizes.
    - **Binary Data**: It is particularly suitable for binary data, where elements are either present or absent in the sets. It can be applied to scenarios where the presence or absence of specific features or attributes is important for comparison.
- **Applications**
    - In the context of a recommender system, Jaccard similarity can be used to identify users with similar item preferences and recommend items that are highly rated or popular among those similar users. By leveraging Jaccard similarity, the recommender can enhance the personalisation of recommendations and help users discover relevant items based on the preferences of users with similar tastes.
    - Assessing similarity between sets of items liked or interacted with by users.

4. **Euclidean Distance:**

- **Definition:** Represents the straight-line distance between two points in a multidimensional space.
- **Application:** Quantifying the dissimilarity or proximity between user or item vectors.

5. **Manhattan Distance:**

- **Definition:** Measures the distance between two points by summing the absolute differences along each dimension.
- **Application:** Similar to Euclidean distance, but may be less sensitive to outliers.

6. **Hamming Distance:**

- **Definition:** Measures the number of positions at which corresponding bits differ in two binary strings.
- **Application:** Suitable for comparing binary user profiles or item representations.

Choosing the appropriate data mining method and similarity measure depends on the characteristics of the data, the nature of the recommendation problem, and computational considerations. Hybrid approaches that combine multiple methods or measures often yield more robust and accurate recommendations.

DIMENSIONALITY REDUCTION:

Overview:

Dimensionality reduction is a technique used to reduce the number of features (dimensions) in a dataset while preserving its essential information. In the context of recommender systems, dimensionality reduction is often applied to user-item interaction matrices to capture latent factors that represent hidden patterns in the data. By reducing the dimensionality, the computational complexity decreases, and the model becomes more efficient.

Methods:

- **Principal Component Analysis (PCA):** PCA is a popular linear dimensionality reduction method that transforms the original features into a new set of uncorrelated variables (principal components) while preserving the variance in the data.
- **Singular Value Decomposition (SVD):** SVD is a matrix factorization technique that decomposes a matrix into three other matrices, capturing latent factors. It is commonly used in collaborative filtering for recommender systems.
- **Non-Negative Matrix Factorization (NMF):** NMF decomposes a matrix into two lower-rank matrices with non-negative elements, making it suitable for scenarios where non-negativity is a meaningful constraint.

Applications in Recommender Systems:

- **Reducing Sparsity:** Recommender system datasets are often sparse, with many missing values in the user-item interaction matrix. Dimensionality reduction helps in filling in missing values by approximating the original matrix with lower-rank approximations.
- **Capturing Latent Factors:** By reducing the dimensionality, latent factors representing user preferences and item characteristics can be identified, leading to more efficient and effective recommendations.

SINGULAR VALUE DECOMPOSITION:

When it comes to dimensionality reduction, the Singular Value Decomposition (SVD) is a popular method in linear algebra for matrix factorization in machine learning. Such a method shrinks the space dimension from N-dimension to K-dimension (where K<N) and reduces the number of features. SVD constructs a matrix with the row of users and columns of items and the elements are given by the users' ratings. Singular value decomposition decomposes a matrix into three other matrices and extracts the factors from the factorization of a high-level (user-item-rating) matrix.

$$A = USV^T$$

- Matrix U: singular matrix of (user*latent factors)
- Matrix S: diagonal matrix (shows the strength of each latent factor)
- Matrix U: singular matrix of (item*latent factors)

From matrix factorization, the latent factors show the characteristics of the items. Finally, the utility matrix A is produced with shape m*n. The final output of the matrix A reduces the dimension through latent factors' extraction. From the matrix A, it shows the relationships between users and items by mapping the user and item into *r*-dimensional latent space. Vector X_i is considered each item and vector Y_u is regarded as each user. The rating is given by a user on an item as $R_{ui} = X^T_i Y_u$. The loss can be minimized by the square error difference between the product of R_ui and the expected rating.

$$Min(x, y) \sum_{(u, i)\in K} (r_{ui} - x_i^T y_u)^2$$

Regularization is used to avoid overfitting and generalize the dataset by adding the penalty.

$$Min(x, y) \sum_{(u, i)\in K} (r_{ui} - x_i^T y_u)^2 + \lambda(||x_i||^2 + ||y_u||^2)$$

Here, we add a bias term to reduce the error of actual versus predicted value by the model.

(u, i): ***user-item pair***

μ: ***the average rating of all items***

bi: ***average rating of item*** **i** ***minus*** **μ**

bu: ***the average rating given by user*** **u** ***minus*** **μ**

The equation below adds the bias term and the regularization term:

$$Min(x, y, b_i, b_u) \sum_{(u, i)\in K} (r_{ui} - x_i^T y_u - \mu - b_i - b_u)^2 + \lambda(||x_i||^2 + ||y_u||^2 + b_i^2 + b_u^2)$$

Introduction to truncated SVD

When it comes to matrix factorization technique, truncated Singular Value Decomposition (SVD) is a popular method to produce features that factors a matrix M into the three matrices U, Σ, and V. Another popular method is Principal Component Analysis (PCA). Truncated SVD shares similarity with PCA while SVD is produced from the data matrix and the factorization of PCA is generated from the covariance matrix. Unlike regular SVDs, truncated SVD produces a factorization where the number of columns can be specified for a number of truncation. For example, given an n x n matrix, truncated SVD generates the matrices with the specified number of columns, whereas SVD outputs n columns of matrices.

The advantages of truncated SVD over PCA

Truncated SVD can deal with sparse matrix to generate features' matrices, whereas PCA would operate on the entire matrix for the output of the covariance matrix.

1. Hands-on experience of python code
2. Data Description:

The metadata includes 45,000 movies listed in the Full MovieLens Dataset and movies are released before July 2017. Cast, crew, plot keywords, budget, revenue, posters, release dates, languages, production companies, countries, TMDB vote counts and vote averages are in the dataset. The scale of ratings is 1–5 and obtained from the official GroupLens website. The dataset is referred to from the Kaggle dataset.

3. Recommending movies using SVD

Singular value decomposition (SVD) is a collaborative filtering method for movie recommendation. The aim for the code implementation is to provide users with movies' recommendation from the latent features of item-user matrices. The code would show you how to use the SVD latent factor model for matrix factorization.

Applications in Recommender Systems:

- **Matrix Factorization:** SVD is used to factorize the user-item interaction matrix into lower-rank approximations, capturing latent factors that represent user preferences and item characteristics.
- **Collaborative Filtering:** SVD is a key technique in collaborative filtering-based recommender systems, where it helps in identifying latent relationships between users and items.
- **Handling Sparsity:** SVD can handle sparse matrices effectively, providing a way to impute missing values in the original matrix and improving the quality of recommendations.
- **Regularization Techniques:** Regularized versions of SVD, such as Regularized SVD, incorporate regularization terms to prevent overfitting and enhance the generalization ability of the model.